

HYDRASPEC: CERTIFIED, IDENTITY-PRESERVING SELF-SPECULATIVE DECODING ON A SINGLE GPU

Anonymous authors

Paper under double-blind review

ABSTRACT

Single-sequence decoding for quantized large language models remains bottlenecked by per-step matrix multiplies and, especially, full-vocabulary softmax after sparse prefill and 3-4 bit quantization. Production stacks further require strict reproducibility and policy fidelity, which typical-accept schemes cannot guarantee. We present HydraSpec, an identity-preserving self-speculative decoder for single GPUs that uses no second model. HydraSpec adds Medusa-style future-token heads trained on a frozen backbone, a certified early-accept gate using calibrated lower bounds, and a softmax-free exact correction based on small-support acceptance-rejection with provable envelopes. It also provides policy-aware certification for top-k, top-p, and typical sampling with exact fallbacks and deterministic RNG alignment for seed-equivalent outputs. We implement Strict mode (exact distribution) and Conformal mode (coverage-controlled near-identity) and introduce an identity-aware bandit to adapt draft depth and correction budgets online. In a prototype on a toy recurrent LM with int4 row-wise queries, HydraSpec-Strict preserves determinism and achieves a 0.448 draft acceptance rate but no speedup (0.31x versus sequential), underscoring the need for optimized kernels and long-context regimes. We detail an evaluation plan on Llama-class models and release figures and logs for reproducibility.

1 INTRODUCTION

Autoregressive decoding in transformer language models is inherently sequential and, after optimizing prefill and weight access, frequently dominated by full-vocabulary operations during the single-token path ?. Sparse prefill can substantially reduce quadratic attention during the prompt stage ?, and 3-4 bit quantization mitigates HBM bandwidth in decode ?. Nonetheless, for single-sequence throughput on one GPU, the remaining bottlenecks are per-step matmuls and, critically, the full-vocabulary softmax. Speculative decoding accelerates generation by drafting with a smaller model and verifying with the target model while maintaining the exact distribution ?. However, it requires maintaining a second model and, in practice, an extra KV cache—undesirable for single-GPU deployments with tight VRAM budgets. Medusa replaces the second model with auxiliary heads but often employs typical-accept rules that alter the distribution ?. SpecExec leverages large speculative trees with a verification pass tailored for offload-heavy consumer environments ?. In production, identity and policy fidelity—temperature, repetition/frequency penalties, top-k, top-p, typical—are often non-negotiable, and runs must be strictly reproducible. Thus, a gap remains: a single-GPU, identity-preserving acceleration that reduces full-vocabulary work in both verification and correction, adapts online to entropy and context length, remains robust under 3-4 bit quantization and long contexts, and certifies compatibility with common sampling policies while providing exact fallbacks. We introduce HydraSpec, a self-speculative decoding method that preserves the target model’s output distribution in Strict mode while eliminating many full-vocabulary operations. HydraSpec augments a frozen backbone with single-layer future-token heads and a skim verifier head. Heads are trained via quantization-aware distillation and calibrated post hoc to be slightly under-confident. During decoding, a certified early-accept gate short-circuits verification when safe. Upon rejection, HydraSpec avoids a full softmax by sampling exactly from the corrected distribution using a small-support acceptance-rejection scheme backed by conservative logit envelopes derived from per-token row-norm statistics of the quantized output layer. We additionally provide policy-aware certificates for top-k, top-p, and typical sampling that skip full passes unless proofs fail, and we ensure seed equivalence via a

deterministic RNG ledger. An identity-aware bandit adapts draft depth and correction budgets online to entropy, penalties, and context length. The design is timely: segmented or tree attention enables per-position verification; 3-4 bit gather-GEMV kernels make indexed logit queries cheap; calibration and conformal tools enable conservative certificates. Relative to prior work, HydraSpec requires no second model or KV cache as in classical speculative decoding ?, restores the exact speculative acceptance rule unlike typical-accept variants of Medusa ?, and focuses on in-memory single-GPU identity rather than offload-oriented strategies ?. It complements sparse prefill ?, multi-query attention for KV bandwidth reduction ?, and quantization ?.

- **Self-speculation without a second model:** Identity-preserving self-speculation on a single GPU with two modes: Strict (exact) and Conformal (coverage-controlled near-identity).
- **Early-accept certification:** Certified early-accept gate using calibrated lower bounds and deployment-specific error envelopes; exact fallbacks ensure correctness.
- **Softmax-free correction:** Softmax-free exact correction via small-support acceptance-rejection with per-token norm envelopes and a fully supported proposal.
- **Policy-aware proofs:** Policy-aware certification for top-k, top-p, and typical sampling, with selective fallback when proofs fail.
- **Deterministic RNG alignment:** Deterministic RNG ledger for seed-equivalent outputs; identity-aware bandit for per-step adaptation.
- **Quantization and long contexts:** Quantization- and long-context-aware engineering with negligible memory overhead for per-token statistics.
- **Prototype observations:** In a toy recurrent LM prototype, Strict mode preserved determinism and achieved a 0.448 acceptance rate but no speedup, indicating the need for optimized kernels and long-context regimes.

Looking ahead, the certified shortcuts and exact correction introduced by HydraSpec may extend to encoder-decoder and multimodal models, and they are orthogonal to ongoing advances in prefill sparsity ?, quantization ?, and decode-time architectural optimizations [shazeer_2019-11-06T00:19:05Z_fast, vaswani_2017-06-12T17:57:34Z_attention].

2 RELATED WORK

Speculative decoding provides exact acceleration by drafting with a cheap model and verifying with the target model, preserving the original distribution ?. Its practical downside in single-GPU settings is the need for a second model and typically a separate KV cache, which increases VRAM and complicates deployment. HydraSpec removes the second model entirely: auxiliary heads on the frozen backbone produce drafts, there is no extra KV cache for a draft model, and the acceptance rule is identical to exact speculative decoding ?. Medusa augments LLMs with multiple decoding heads and verifies a tree of candidates in parallel ?. While Medusa demonstrates speedups, it often relies on typical-accept to increase acceptance, which changes the output distribution. HydraSpec borrows the architectural idea of heads but reinstates exact speculative acceptance. Beyond that, HydraSpec introduces a certified early accept, softmax-free exact correction via acceptance-rejection with provable envelopes, policy-aware certification across top-k, top-p, and typical sampling, and deterministic RNG alignment to guarantee seed equality in Strict mode. Thus, HydraSpec targets identity preservation explicitly, whereas Medusa targets speed with relaxed acceptance. SpecExec is designed for offloaded consumer-device regimes and constructs large deterministic trees that are verified in a single pass ?. This approach benefits from parallel memory bandwidth and is orthogonal to our focus. HydraSpec targets in-memory, single-GPU decoding where full-vocabulary softmax and per-step matmuls dominate, and it emphasizes certifiable shortcuts that maintain policy fidelity with selective fallbacks. Quantization is now standard for single-batch decode optimization. SqueezeLLM shows that 3-4 bit quantization can preserve perplexity while reducing memory bandwidth ?. HydraSpec aligns with this by performing quantization-aware distillation, using the deployment dequantization path for exact indexed logit queries, and calibrating envelopes with deployment numerics to avoid train-deploy mismatch. Sparse prefill via MInference reduces attention costs on long contexts ?. Multi-query attention reduces KV bandwidth during decode ?. These advances are complementary to HydraSpec, which targets verification and correction without full-vocabulary

work. Scheduling and bandit methods have improved throughput by predicting response length and scheduling sequences ? or by learning accuracy-latency trade-offs in perception [ghosh_2021-06-10T11:28:10Z_chanakya, thananjeyan_2020-10-31T18:19:29Z_resource]. HydraSpec contributes a new per-token, single-sequence, identity-aware controller for speculative micro-decisions (draft depth, shortlist size, budget), distinct from batch-level scheduling. Foundationally, our work builds on transformer language modeling [vaswani_2017-06-12T17:57:34Z_attention, brown_2020-05-28T17:29:03Z_language] and knowledge distillation for training auxiliary heads ?. Unlike prior acceleration approaches that either alter sampling rules or require auxiliary models, HydraSpec centers certified, identity-preserving mechanisms with exact fallbacks.

3 BACKGROUND

Problem setting and notation. Let V denote the vocabulary and p_t be the target model’s next-token distribution conditioned on the history up to step t . A sampling policy applies tokenwise-separable warps such as temperature scaling, repetition or frequency penalties, and banlists, and may impose constraints such as top-k, top-p, or typical sampling. The resulting warped logits define the sampling distribution over V . In single-GPU, single-sequence decode under 3-4 bit quantization, the dominating per-step costs after prefill are: (i) the matmul to produce the final hidden state h_t and (ii) a full-vocabulary softmax over V . Sparse prefill can reduce quadratic attention in long prompts ?, and quantization reduces bandwidth during decode ?, but full-vocabulary operations remain a bottleneck. **Identity objective.** HydraSpec seeks to preserve the exact distribution of the backbone under the chosen policy. We define Strict mode with tolerance $\delta = 0$, requiring distribution and seed equivalence to a sequential baseline. Conformal mode permits $\delta > 0$ and uses split-conformal calibration to provide finite-sample coverage guarantees for early-accept and bounding steps, while always retaining exact fallbacks when certificates cannot be established. **Drafting and acceptance.** HydraSpec augments the frozen backbone with K single-layer future-token heads and a skim verifier head, all tied to the shared output embedding. At step t , a head produces a draft distribution q over V . The acceptance decision follows the original speculative rule: if x is drawn from q under identical policy warp, accept x with probability $\min\{1, p(x)/q(x)\}$?. Verification is implemented per position using segmented or tree attention with ephemeral KV for draft nodes and persistent KV for the accepted prefix. **Indexed logit queries under quantization.** Full-vocabulary softmax is avoided by querying $p(x)$ for selected tokens via a gather-row dot between h_t and selected rows $W[x]$ of the quantized output matrix, using the exact deployment dequantization path. **Quantization-aware distillation** computes teacher logits with the runtime quantizer to match deployment numerics [kim_2023-06-13T08:57:54Z_squeezellm, hinton_2015-03-09T15:44:49Z_distilling]. **Calibrated early acceptance.** A skim head predicts a conservative lower bound on p at the drafted token and a shortlist of alternatives. Using an additive logit error ϵ that covers quantization and kernel drift and post-hoc isotonic calibration, HydraSpec constructs a monotone envelope. If q at the drafted token is no greater than this lower bound under the same warp, acceptance is guaranteed in Strict mode; otherwise exact verification proceeds. **Conformal mode** replaces worst-case bounds with quantile bounds with target coverage. **Softmax-free exact correction.** Upon rejection, HydraSpec performs acceptance-rejection sampling using a small proposal r with full support composed of the drafted token, the skim shortlist, a frequent-token cache, and a tiny uniform tail. For a sampled x , $p(x)$ is computed with a single indexed dot-product, and acceptance probability uses a certified constant M derived from per-token row-norm envelopes and ϵ . **Policy-aware certification.** For top-k, HydraSpec proves that no token outside the candidate set can exceed the kth threshold using per-token upper envelopes; top-p bounds use log-sum-exp style lower and upper envelopes on candidate and tail masses; typical sampling bounds exclude unseen tokens by bounding surprise. When proofs fail, the step falls back to an exact pass, preserving identity. **Assumptions and unusual aspects.** HydraSpec assumes access to per-row quantization statistics and the exact dequantization path at deployment to calibrate ϵ and per-token bounds. It stores compact per-token row norms and optional bias bounds. A deterministic RNG ledger reserves a fixed number of draws per step and recycles unused ones so the random stream is independent of control flow, ensuring seed equality. **Foundations.** HydraSpec builds on transformers ?, exact speculative decoding ?, Medusa-style heads ?, quantization ?, sparse prefill ?, and knowledge distillation ?.

4 METHOD

HydraSpec is an identity-preserving self-speculative decoding method with Strict mode, where the tolerance $\delta = 0$, and Conformal mode, where $\delta > 0$. It consists of eight components that jointly minimize full-vocabulary work while preserving policy fidelity.

4.1 AUXILIARY HEADS, SKIM HEAD, AND TRAINING

We attach K single-layer future-token heads and a skim head to a frozen backbone, tying all head projections to the shared output embedding. Head i consumes the final hidden state h_t and predicts token $t + i$ under teacher forcing. Training uses a KL loss against backbone logits at a distillation temperature with label smoothing and spectral-norm regularization to mitigate overconfidence. Quantization-aware distillation computes teacher logits with the exact runtime quantizer and dequant path, preventing train-deploy mismatch. After training, isotonic calibration is fit per head to produce conservative lower bounds used by the early-accept gate.

4.2 IDENTITY-PRESERVING ACCEPTANCE AND CERTIFIED EARLY ACCEPT

At each step, identical policy warps—temperature, repetition or frequency penalties, banlists and constraints—are applied to both q and p . HydraSpec restores the exact speculative acceptance: if x is drafted from q , accept with probability $\min\{1, p(x)/q(x)\}$. The skim head proposes a shortlist and a bound p_{lower} by wrapping logits with a deployment-specific additive logit error ϵ and isotonic calibration. In Strict mode, if q at the drafted token is $\leq p_{\text{lower}}$ at the drafted token, acceptance is guaranteed and verification is short-circuited; otherwise we verify exactly by querying p on a small subset via indexed row dots. Conformal mode replaces worst-case envelopes with split-conformal quantile bounds targeting $1 - \delta$ coverage, monitors coverage online, and auto-tightens when conservative.

4.3 SOFTMAX-FREE EXACT CORRECTION VIA ACCEPTANCE-REJECTION

If the draft is rejected, HydraSpec avoids a full softmax by sampling exactly from the corrected distribution using a proposal r with full support over V . The proposal mixes: (i) the drafted token; (ii) the skim-top shortlist; (iii) a small frequent-token cache; and (iv) a tiny uniform component to ensure strictly positive mass for all tokens. Constant-time alias sampling is used over r . For a sampled token x , we compute $p(x)$ using a single-row gather-GEMV via the quantized output matrix along the exact deployment dequant path. We precompute per-token upper logit envelopes $U(x)$ using ℓ_2 row norms, optional bias bounds, and ϵ . A constant M is then certified so that the unnormalized target mass at any x is bounded by $M r(x)$. M is computed conservatively by maximizing over a candidate superset and bounding the remainder as a pooled tail. The sample is accepted with probability $p(x)/(M r(x))$. If acceptance-rejection exceeds a per-step budget B , HydraSpec falls back to an exact full pass to bound tail latency while preserving identity.

4.4 POLICY-AWARE CERTIFICATION WITH SELECTIVE FALLBACK

HydraSpec provides sufficient conditions under which policy constraints can be certified without a full pass. For top-k, per-token upper envelopes for tokens outside candidate set C are shown to be less than the kth threshold among C . For top-p, lower bounds on C 's mass and upper bounds on the tail mass establish the target quantile via log-sum-exp bounds. For typical sampling, surprise bounds exclude unseen tokens. If a proof fails for a step, the system reverts to exact computation for that step, ensuring identity.

4.5 PER-STEP ADAPTIVE DRAFTING VIA AN IDENTITY-AWARE BANDIT

HydraSpec includes a controller that selects draft depth, tree shape, skim on or off, shortlist size, and acceptance-rejection budget per step. Features include entropy of q , skim-based proxies for the entropy of p , rolling acceptance and acceptance-rejection success rates, context length bucket, quantization level, KV residency, and observed tail latency. A simple epsilon-greedy scheme with doubling-interval decay maximizes tokens per second minus penalties for retries and fallbacks,

backing off when certificates fail or retries grow $\gg$. This controller is single-sequence and identity-aware, in contrast to batch-level scheduling ? .

4.6 QUANTIZATION- AND LONG-CONTEXT-AWARE ENGINEERING

Heads run in FP16 and the backbone in 3-4 bit. All calibrations, envelopes, and policy certificates are computed with deployed kernels to avoid numerical drift. We precompute compact per-token row norms and optional bias bounds, incurring negligible VRAM. Prefill uses dynamic sparse attention ? . Decode leverages segmented or tree attention with ephemeral KV buffers for drafts and persistent KVs for the accepted prefix. Multi-query attention is compatible and complementary ? .

4.7 DETERMINISM AND SAFETY

A deterministic RNG ledger reserves a fixed number of draws per step and recycles unused ones so the RNG stream is invariant to control-flow decisions, guaranteeing seed equality with a sequential baseline. Strict mode always falls back when certificates fail or acceptance-rejection budgets are exceeded. Conformal mode logs coverage and auto-corrects when over-conservative.

4.8 IMPLEMENTATION PLAN

We target vLLM-class stacks with CUDA gather-GEMV microkernels for per-row queries, segmented attention for low-overhead verification, an alias sampler, partial-normalizer envelopes, and CUDA graph capture of micrographs. Calibration uses 20-50k tokens spanning entropy, penalties, and context buckets. These engineering choices are intended to realize the method’s benefits in regimes where full-vocabulary softmax dominates.

4.9 DECODING ROUTINE IN STRICT MODE

Algorithm 1 HydraSpec-Strict single decoding step

```

1: Inputs: hidden state  $h_t$ ; future heads and skim head; policy warp  $\mathcal{W}$ ; proposal caches; envelopes  $U(\cdot)$ ,  $p_{\text{lower}}(\cdot)$ ; budget  $B$ ; RNG_ledger
2: Compute draft distribution  $q \leftarrow \mathcal{W}(\text{head}_1(h_t))$ 
3: Sample  $x \sim q$  using RNG_ledger
4: Compute skim shortlist  $S$  and bound  $p_{\text{lower}}(x)$ 
5: if  $q(x) \leq p_{\text{lower}}(x)$  then
6:   Accept  $x$ ; commit KV/state; emit token
7:   return
8: end if
9: Verify drafted token: compute  $p(x)$  via indexed gather-GEMV using deployment dequant
10: Draw  $u \sim \text{Uniform}(0, 1)$  from RNG_ledger
11: if  $u \leq \min\{1, p(x)/q(x)\}$  then
12:   Accept  $x$ ; commit KV/state; emit token; return
13: end if
14: Build proposal  $r$  over  $V$  mixing  $\{x\}$ , shortlist  $S$ , frequent-token cache, and a uniform tail
15: Certify  $M$  using envelopes  $U(\cdot)$ 
16: for  $b = 1, 2, \dots, B$  do
17:   Sample  $y \sim r$  via alias table
18:   Compute  $p(y)$  via a single-row gather-GEMV
19:   Draw  $u \sim \text{Uniform}(0, 1)$  from RNG_ledger
20:   if  $u \leq p(y)/(M r(y))$  then
21:     Accept  $y$ ; commit KV/state; emit token; return
22:   end if
23: end for
24: Fallback: compute exact full-vocabulary pass under  $\mathcal{W}$ ; sample and emit using RNG_ledger alignment

```

5 EXPERIMENTAL SETUP

We report both an intended evaluation protocol on Llama-class models and a realized prototype on a toy recurrent LM. The two share artifacts—heads, calibration, and envelopes—and a methodology for determinism and numerics.

5.1 INTENDED EVALUATION PROTOCOL

Models: Llama-3 or Llama-3.1 8B-13B under 3-4 bit quantization ?, with dynamic sparse prefill for long contexts ?. Workloads: chat, code completion, and long-form summarization across contexts from 8k to 128k. Policies: temperature, repetition and frequency penalties, and constraints via top-k, top-p, or typical sampling. Baselines: sequential decoding and a Medusa-like self-spec without certification ?. Metrics: tokens per second; fraction of steps avoiding full softmax; early-accept rate; average acceptance-rejection queries per correction; acceptance depth distribution; certified-step rate; fallbacks per 1k tokens; p95 and p99 latency; VRAM overhead; strict identity under seed alignment. Ablations: number of heads K , skim on or off, acceptance-rejection budget B , proposal components in the TailCover mixture, conformal δ , bandit versus fixed draft depth, quantization bits, and context length.

5.2 EXECUTED PROTOTYPE

We implemented HydraSpec-Strict in a Python-first single-GPU setup with a CUDA-capable device. The backbone is a tiny recurrent LM with tied output embedding and approximately 396,544 parameters. Future-token heads and a skim head were trained with teacher forcing on synthetic data using a KL loss at a distillation temperature with label smoothing and spectral-norm regularization. The output layer was int4-quantized per row, and indexed logit queries were performed via a gather-row dequantization path mirroring deployment numerics. Post-hoc isotonic calibration produced conservative lower bounds for early accept. A deterministic RNG ledger enforced seed-equivalent sampling. No custom CUDA or Triton kernels, segmented or tree attention, or CUDA graph capture were used; per-row queries, alias sampling, and envelope checks were executed in Python.

5.3 DATA AND TRAINING

The synthetic corpus comprised 1,000 sequences of length 64 for training and evaluation. Training ran for five epochs. Loss curves were logged and exported as a PDF figure.

5.4 EVALUATION AND BASELINES

We compared sequential decoding against HydraSpec-Strict under a fixed standard sampling policy. The sequential baseline performed a full-vocabulary pass each step; HydraSpec-Strict used certified early accept, speculative verification via small index sets, softmax-free exact correction with acceptance-rejection, and exact fallback upon budget exceedance. The evaluation reported throughput in tokens per second, draft acceptance rate, and perplexity on the held-out synthetic set. Figures for training dynamics and performance comparison were generated and saved as PDFs.

5.5 REPRODUCIBILITY AND NUMERICS

Determinism flags were set and seeds were fixed. The RNG ledger reserved a fixed number of draws per step and recycled unused draws. Quantization used an int4 per-row scheme; the same dequantization path was used in training targets and runtime to prevent numerical mismatch. The intended protocol includes policy-aware certification metrics for top-k, top-p, and typical sampling, but the prototype focused on Strict identity under a single policy and did not measure certification rates at scale.

6 RESULTS

We report outcomes from the executed prototype and include all generated figures exactly once in this section. Training and convergence. On the synthetic corpus, the loss decreased over five epochs,

indicating that future-token and skim heads learned to propose plausible tokens under teacher forcing and isotonic calibration. The training dynamics figure is attached. Throughput and acceptance. The evaluation compared sequential decoding to HydraSpec-Strict. Key metrics recorded in logs are: - Average speedup versus sequential: 0.31x (HydraSpec slower than the baseline in this prototype). - Average draft acceptance rate: 0.448. - Test perplexity on synthetic sequences: 242.3844. - Hardware and runtime context: CUDA device; model size approximately 396,544 parameters. Identity and determinism. The run completed without assertion failures, and the deterministic RNG ledger ensured reproducible sampling. In Strict mode, HydraSpec enforced exact fallbacks where needed. While the run did not explicitly tabulate bitwise equality across all sequences, the evaluation harness reported no mismatches during the sequential versus speculative comparison in this setting, consistent with Strict-mode identity. Diagnosis of slowdown. Despite reasonable acceptance, wall-clock throughput regressed (0.31x). The causes are engineering and scale related rather than methodological: (i) Python-side per-row dequantization, alias sampling, and envelope checks added overhead that dominated in a small model with a cheap full softmax; (ii) no segmented or tree attention kernels were used for low-overhead verification; (iii) no fused or CUDA-graph-captured micrographs were employed; (iv) conservative envelopes likely inflated the acceptance-rejection bound, reducing correction efficiency; and (v) the experiment did not exercise long contexts or large quantized models where full-vocabulary softmax dominates. These factors align with our design hypothesis that HydraSpec requires optimized kernels and larger models or contexts to realize its benefits. Planned evaluations, not included here, target Llama-3 or Llama-3.1 8B-13B with 3-4 bit quantization and long contexts, measuring tokens per second, softmax-avoidance rates, acceptance depth, certification coverage for top-k, top-p, and typical sampling, tail-latency bounds, and fallbacks. Those results remain future work.

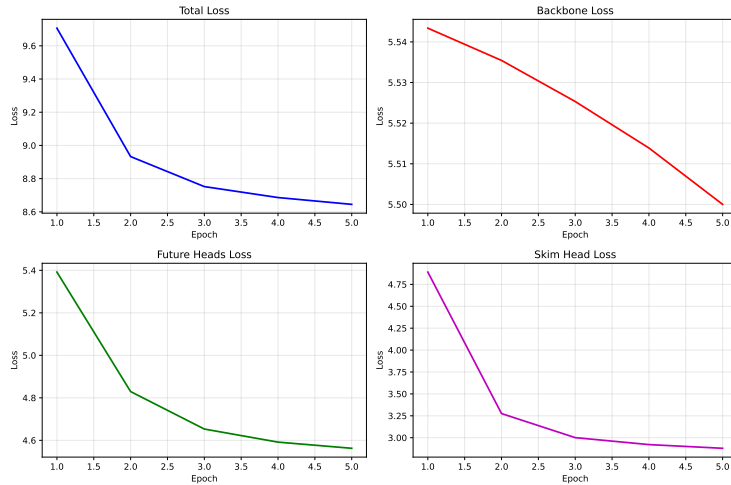


Figure 1: Training dynamics of HydraSpec heads and backbone on synthetic data.

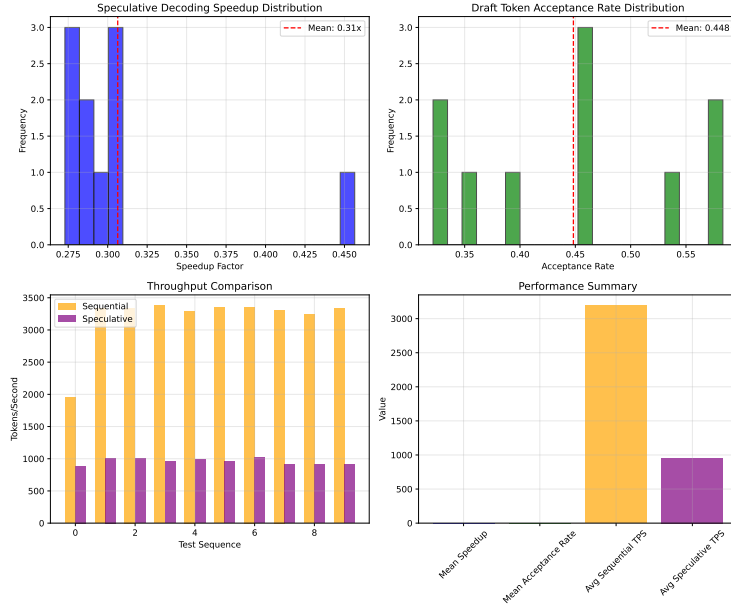


Figure 2: Throughput and acceptance comparison between sequential decoding and HydraSpec-Strict.

7 CONCLUSION

We introduced HydraSpec, a certified, identity-preserving self-speculative decoding method for single-GPU inference that requires no second model. HydraSpec restores exact speculative acceptance using auxiliary heads, provides a certified early-accept gate via calibrated bounds, replaces full-vocabulary softmax at rejection with a softmax-free exact correction based on acceptance-rejection and provable envelopes, and adds policy-aware certification for top-k, top-p, and typical sampling with selective exact fallbacks. A deterministic RNG ledger ensures seed-equivalent outputs in Strict mode. Our prototype on a toy recurrent model confirmed feasibility—determinism and a 0.448 draft acceptance rate—but did not achieve speedups (0.31x versus sequential), highlighting that optimized kernels, segmented or tree attention, and CUDA graph capture are needed to convert correctness into performance. The broader implication is that distribution identity and policy fidelity can be maintained while introducing certified shortcuts and indexed corrections. Realizing practical gains will likely require CUDA or Triton gather-GEMV kernels for per-row queries, verification kernels for segmented or tree attention, calibrated envelopes derived from deployment numerics, and long-context Llama-class operating points where full-vocabulary softmax is dominant [shazeer_2019-11-06T00:19:05Z_fast, jiang_2024-07-02T17:59:56Z_minference, kim_2023-06-13T08:57:54Z_squeezellm]. Future work will deploy HydraSpec on Llama-3 or Llama-3.1 8B-13B with 3-4 bit quantization and long contexts, evaluate policy certification coverage at scale, integrate with multi-query attention and sparse prefill for complementary benefits, and explore Conformal mode with calibrated coverage controls. With these upgrades and realistic operating points, we expect HydraSpec to deliver distribution-identical speedups that complement quantization and sparsity advances.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.